

LAB ASSIGNMENT – 1.5

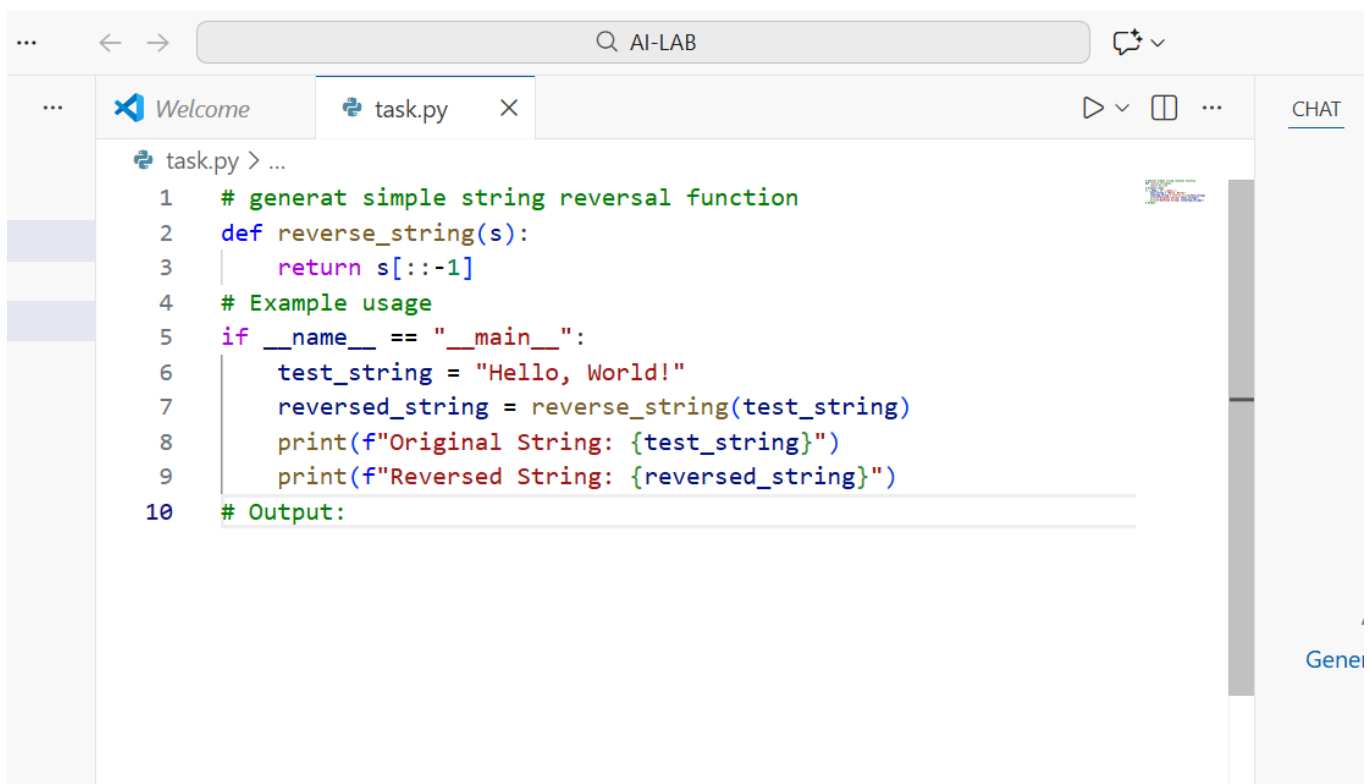
HALLTICKET NO : 2303A51416

BATCH NO : 29

TASK - 01

PROMPT : Write a python program to reverse a string without using functions

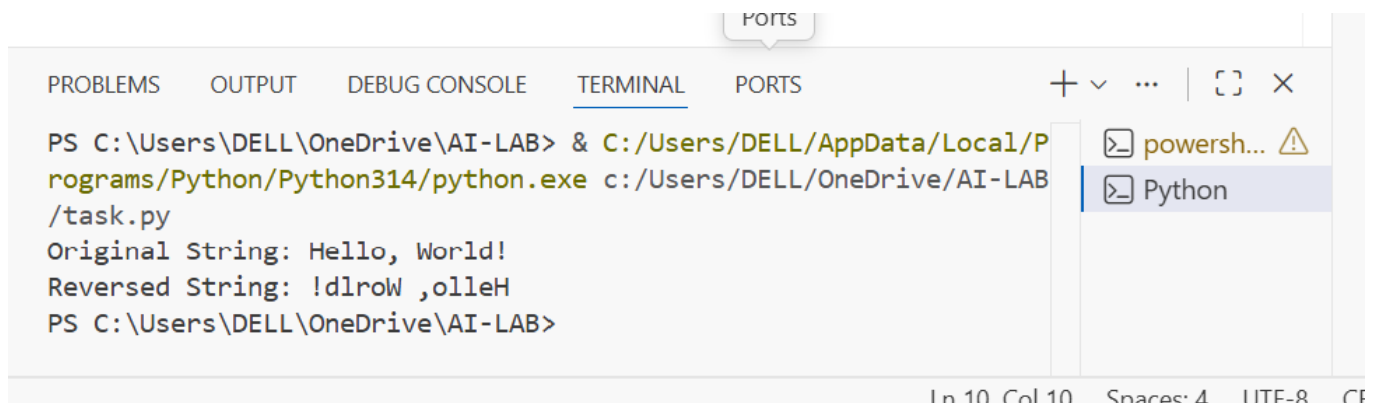
CODE :



The screenshot shows a code editor with a file named 'task.py'. The code defines a function 'reverse_string(s)' that returns the reversed string using slicing. It then uses this function to reverse the string 'Hello, World!' and prints both the original and reversed strings.

```
task.py > ...
1  # generat simple string reversal function
2  def reverse_string(s):
3      return s[::-1]
4  # Example usage
5  if __name__ == "__main__":
6      test_string = "Hello, World!"
7      reversed_string = reverse_string(test_string)
8      print(f"Original String: {test_string}")
9      print(f"Reversed String: {reversed_string}")
10 # Output:
```

OUTPUT :



The screenshot shows a terminal window with the command prompt 'PS C:\Users\DELL\OneDrive\AI-LAB>'. The command executed is 'C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py'. The output shows the original string 'Hello, World!' and the reversed string '!dlroW ,olleH'.

```
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
Original String: Hello, World!
Reversed String: !dlroW ,olleH
PS C:\Users\DELL\OneDrive\AI-LAB>
```

OBSERVATION:

It was observed that GitHub Copilot was able to generate a correct Python program to reverse a string without using any user-defined functions. The program accepted user input and implemented the reversal logic directly within the main code, meeting the task requirements. The generated logic was simple and easy to understand, making it suitable for beginners and small-scale applications. However, since the logic was written procedurally without modularization, the code lacked reusability and would become harder to maintain or extend if the same functionality were required in multiple parts of an application. Overall, this task shows that Copilot is effective in quickly producing basic procedural code, but such an approach is less suitable for larger or more complex software systems where modular design is preferred.

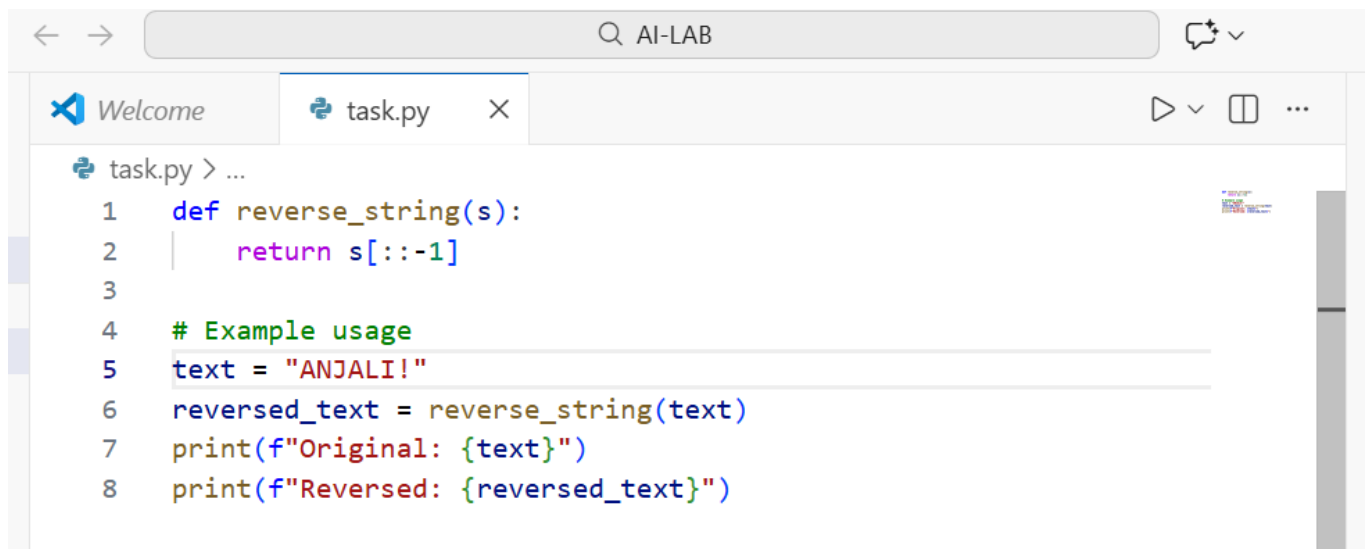
TASK - 02

PROMPT :

Simplify this string reversal code

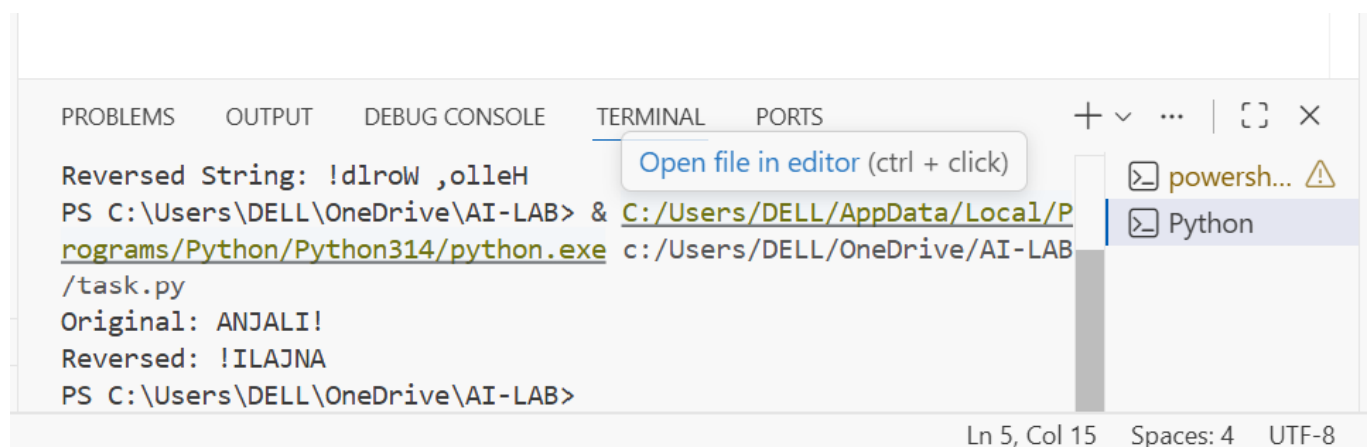
Improve readability and efficiency

Optimize this code without using function



```
task.py > ...
1  def reverse_string(s):
2      return s[::-1]
3
4  # Example usage
5  text = "ANJALI!"
6  reversed_text = reverse_string(text)
7  print(f"Original: {text}")
8  print(f"Reversed: {reversed_text}")
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Reversed String: !dlrow ,olleH
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
Original: ANJALI!
Reversed: !ILAJNA
PS C:\Users\DELL\OneDrive\AI-LAB>
```

OBSERVATION:

it was observed that GitHub Copilot effectively improved the initial procedural string reversal code when prompted to optimize and enhance readability. By simplifying the logic and removing unnecessary variables and loop-based string concatenation, the optimized version became shorter, clearer, and easier to understand for other developers during code review. Copilot suggested using Python's built-in slicing method, which not only reduced the number of lines of code but also improved efficiency. The time complexity was reduced from

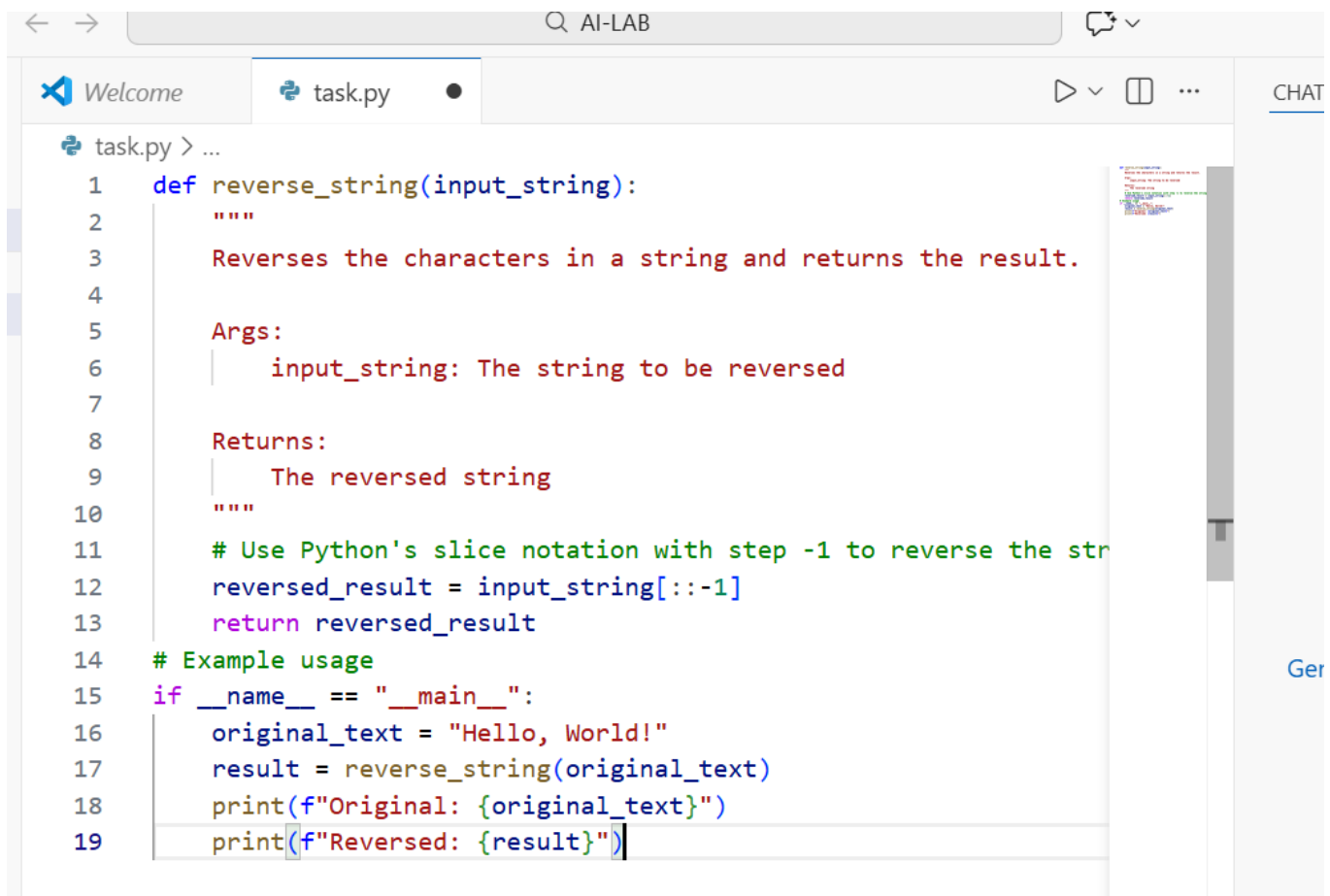
quadratic time in the original version, caused by repeated string concatenation inside a loop, to linear time in the optimized version, as slicing processes the string in a single pass. Overall, this task demonstrates that Copilot can assist in refactoring code to make it more readable, efficient, and suitable for collaborative development environments.

TASK – 03

PROMPT:

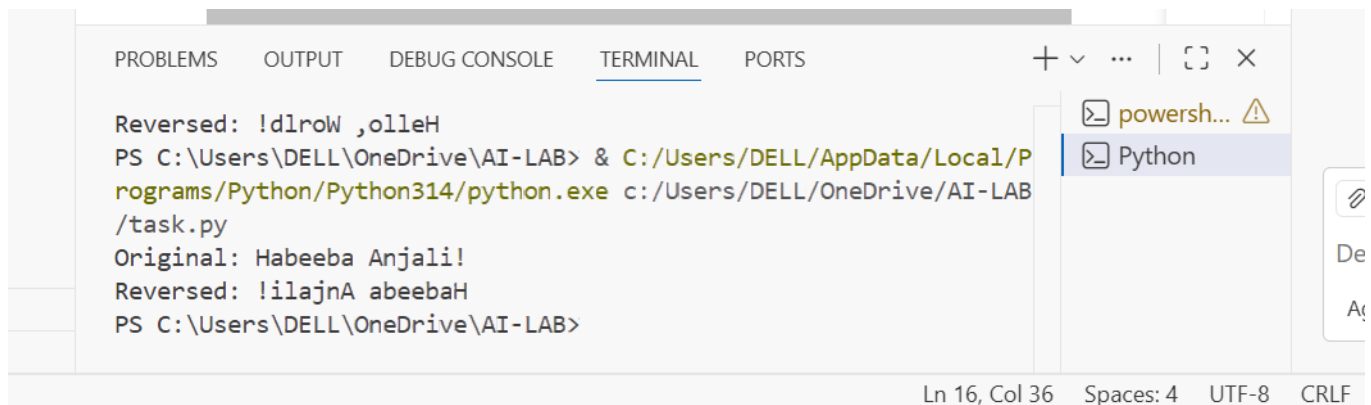
Create a Python function that reverses a string and returns it

Use clear variable names and add meaningful comment

A screenshot of a code editor window. The title bar shows a search icon, the text 'AI-LAB', and a refresh icon. The editor has two tabs: 'Welcome' and 'task.py'. The 'task.py' tab is active, showing a Python script. The script defines a function 'reverse_string' with a docstring that explains its purpose and arguments. It then uses Python's slice notation to reverse the string and returns the result. Finally, it includes an example usage block with a main function that tests the reverse_string function with the string 'Hello, World!'.

```
task.py > ...
1  def reverse_string(input_string):
2      """
3      Reverses the characters in a string and returns the result.
4
5      Args:
6      |   input_string: The string to be reversed
7
8      Returns:
9      |   The reversed string
10     """
11     # Use Python's slice notation with step -1 to reverse the str
12     reversed_result = input_string[::-1]
13     return reversed_result
14
15 # Example usage
16 if __name__ == "__main__":
17     original_text = "Hello, World!"
18     result = reverse_string(original_text)
19     print(f"Original: {original_text}")
20     print(f"Reversed: {result}")
```

OUTPUT:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the output of a Python script that reverses a string. The command executed is `& C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py`. The output shows the original string 'Habeeba Anjali!', its reversed version '!ilajnA abeebaH', and the reversed version of the reversed string, which is the original string 'Habeeba Anjali!'. The terminal also shows the path `PS C:\Users\DELL\OneDrive\AI-LAB>`. On the right side of the terminal, there is a dropdown menu with 'powersh...' and 'Python' options. The status bar at the bottom indicates 'Ln 16, Col 36', 'Spaces: 4', 'UTF-8', and 'CRLF'.

```
Reversed: !dlroW ,olleH
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
Original: Habeeba Anjali!
Reversed: !ilajnA abeebaH
PS C:\Users\DELL\OneDrive\AI-LAB>
```

Ln 16, Col 36 Spaces: 4 UTF-8 CRLF

OBSERVATION:

it was observed that GitHub Copilot successfully generated a function-based Python program to reverse a string, addressing the need for modular design. The use of a user-defined function allowed the string reversal logic to be clearly separated from the main program flow, making the code more structured and easier to understand. Copilot also added meaningful comments that explained the purpose of the function and the steps involved in reversing the string, which improves readability and supports future maintenance. By returning the reversed string from the function, the logic became reusable and could be easily called from multiple parts of an application. The generated program produced correct outputs for different test cases, demonstrating that modular design enhances clarity, reusability, and suitability for larger applications.

TASK-04

Short Analytical Report The procedural approach used in Task 1 is suitable for small and simple programs where the logic is used only once. It provides quick results but lacks reusability and scalability.

The modular approach used in Task 3 improves code organization by separating logic into a user-defined function. This makes the program easier to understand, reuse, debug, and maintain. For large-scale applications, the modular approach is preferred as it supports better design practices and long-term maintainability

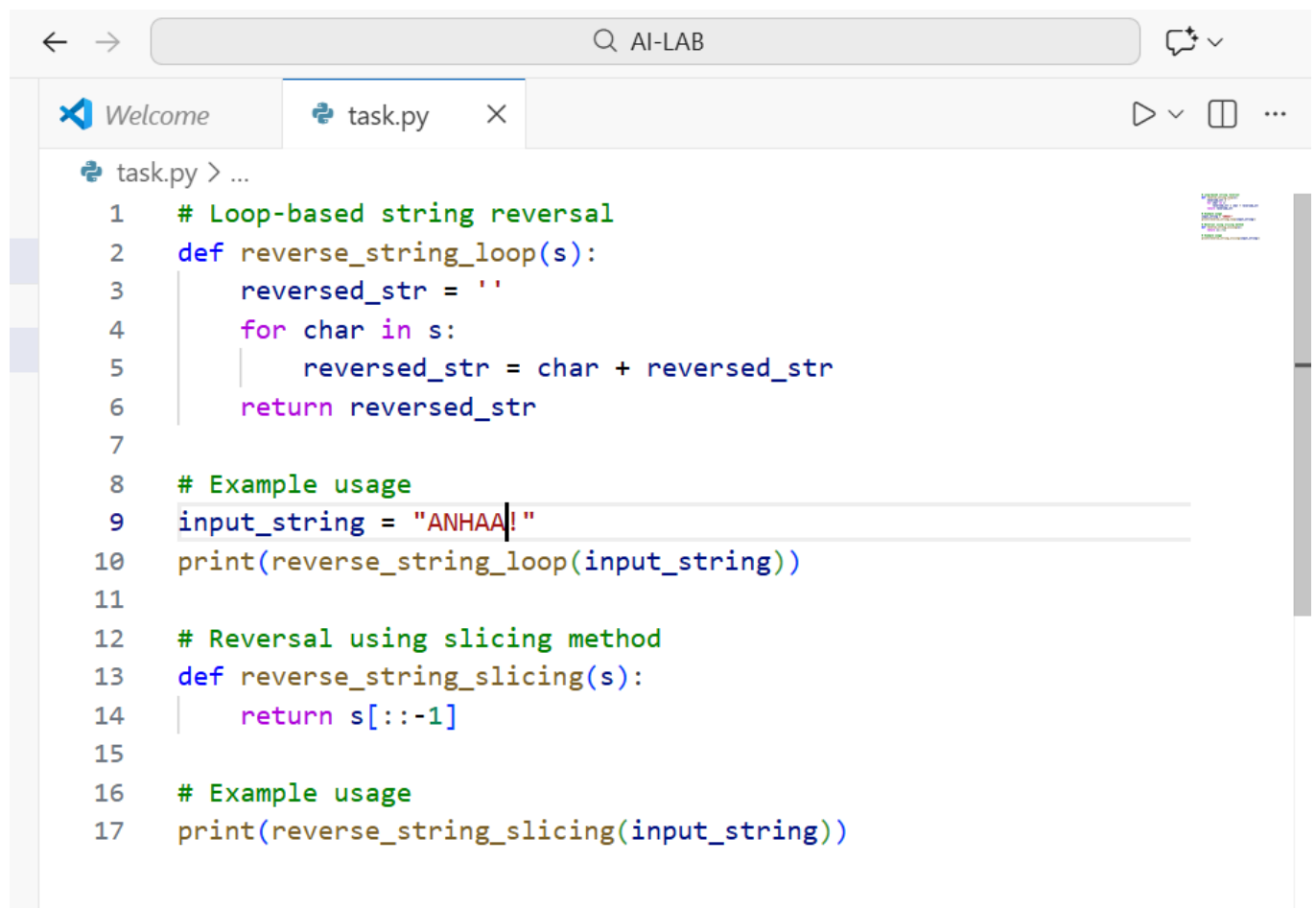
TASK-05

AI-Generated Iterative vs Built-in Approaches

PROMPT:

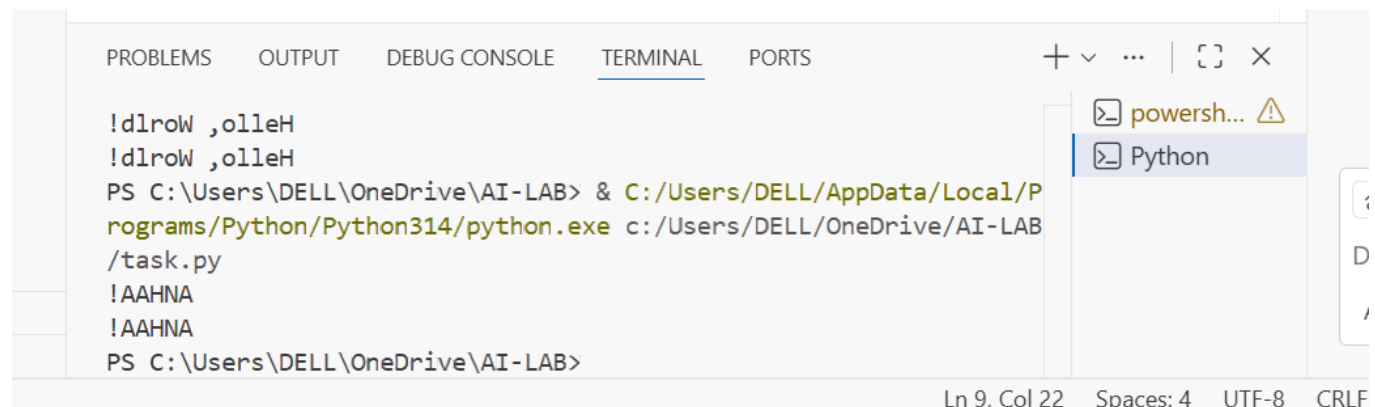
Generate a loop-based string reversal in Python

Reverse a string using Python built-in or slicing method

A screenshot of a code editor interface. The top bar shows a search icon and the text 'AI-LAB'. Below the top bar, there are two tabs: 'Welcome' and 'task.py'. The 'task.py' tab is active. The code editor displays the following Python code:

```
task.py > ...
1  # Loop-based string reversal
2  def reverse_string_loop(s):
3      reversed_str = ''
4      for char in s:
5          reversed_str = char + reversed_str
6      return reversed_str
7
8  # Example usage
9  input_string = "ANHAA!"
10 print(reverse_string_loop(input_string))
11
12 # Reversal using slicing method
13 def reverse_string_slicing(s):
14     return s[::-1]
15
16 # Example usage
17 print(reverse_string_slicing(input_string))
```

Output:



The screenshot shows a Visual Studio Code interface with the 'TERMINAL' tab active. The terminal displays the following output:

```
!dlrow ,olleH
!dlrow ,olleH
PS C:\Users\DELL\OneDrive\AI-LAB> & C:/Users/DELL/AppData/Local/Programs/Python/Python314/python.exe c:/Users/DELL/OneDrive/AI-LAB/task.py
!AAHNA
!AAHNA
PS C:\Users\DELL\OneDrive\AI-LAB>
```

On the right side of the terminal, there is a file explorer showing a folder named 'powersh...' and a file named 'Python'. The status bar at the bottom indicates 'Ln 9, Col 22', 'Spaces: 4', 'UTF-8', and 'CRLF'.

Observation:

it was observed that GitHub Copilot was able to generate multiple correct solutions for the same problem by following different algorithmic paths. When prompted, Copilot produced a loop-based (iterative) string reversal approach that explicitly processes each character and builds the reversed string step by step, making the execution flow easy to follow and suitable for understanding core logic. It also generated a built-in slicing-based approach that relied on Python's optimized features, resulting in shorter, cleaner, and more readable code. The comparison showed that the loop-based method has higher time complexity due to repeated string concatenation and performs less efficiently for large inputs, while the slicing-based approach runs in linear time and offers better performance and scalability. This task demonstrates that AI tools like Copilot can effectively explore alternative logic paths and help developers choose the most appropriate approach based on readability, performance requirements, and application scale.

